

Real-world (a tool backed by a real API call) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/Python), **Advanced** (what a tool backed by a real, changeable service actually needs). Read Basic first even if you already know this — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Your first tool (from the Basic exercise) just did math — no outside call, nothing could really go wrong. A real tool talks to a real service, and real services time out, go down, or send back something unexpected.

You already built a small HTTP client with retry logic back in Doc02. Reuse it here, instead of calling `requests.get(...)` directly inside your tool — the whole point of this exercise is putting that old retry logic somewhere new: behind a tool the model can call.

Pick any free public API that needs no signup (a weather API or a currency-conversion API both work fine).

Things to use:

- `from http_client import request_with_retry` (or however you named Doc02's client — import it, don't rewrite it).
- `@tool from langchain_core.tools`.
- A free API with no signup — for example `https://api.open-meteo.com/v1/forecast` (weather, no key needed).
- Build the URL/params yourself inside the tool function, using the argument the model sent.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Doc02's `request_with_retry()` already handles the messy parts (timeouts, transient failures) — don't rewrite that logic inside the tool function. Instead, call it *from* the tool function, and let the `@tool` decorator wrap the whole thing.

Keep the argument shape small and typed (a single typed parameter like `city: str` is fine to start) — the model needs to know exactly what to send, and Doc02's client needs a real URL built from that argument.

The tool function should stay thin: build the request URL/params, call `request_with_retry(url, params)`, and return the one or two fields the model actually needs — not the whole raw JSON blob. Don't swallow errors here yet — that's next exercise's job. For now, let a real failure actually raise, so you see what an unhandled tool error looks like once.

Write out, in plain words, the full call chain: model decides to call the tool → your tool function runs → it calls `request_with_retry()` → that makes the actual HTTP call → the result flows back up through all three layers.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two things about this tool are demo shortcuts, not production choices — worth naming honestly rather than pretending the exercise's version is finished.

The API's response shape isn't guaranteed to stay the same forever.

`response["current_weather"]["temperature"]` assumes both keys exist. Real public APIs occasionally change their response shape, or return a differently-shaped error body on a bad request instead of the success shape you coded against. A `KeyError` from a missing nested key is a genuine tool crash — no different in effect from the `ZeroDivisionError` the next exercise deliberately teaches you to catch. The real design question: should this tool's happy-path code defend against a malformed response now, or is "let it crash, and let the next exercise's failure-handling wrap it" an acceptable answer for *this* exercise specifically? (It's acceptable here — Real-world's job is proving the plumbing works — but you should be able to say *why* out loud, not just leave it unhandled by accident.)

The hardcoded `CITY_COORDINATES` lookup table doesn't scale. It has exactly 2 cities in it. A tool description that says "get the weather for a named city" is implicitly promising *any* city — but the implementation only actually supports the ones in your dictionary. That's a real gap between what the tool's description tells the model and what the code can actually do — the same category of problem as a vague description, just on the "can it actually do what it claims" side instead of the "does the model understand it" side. A real version either accepts latitude/longitude directly (pushing the city-to-coordinates problem onto the caller) or makes a second real API call to a geocoding service first.

The extra pieces needed to see both of these:

- `response.get("current_weather", {}).get("temperature")` — safe nested access that returns `None` instead of raising, so you can detect and report "the API's response didn't look like I expected" as a clear string, rather than crashing.
- A second `request_with_retry()` call to a free geocoding endpoint (e.g. Open-Meteo's own geocoding API needs no key either), turning a city name into coordinates for *any* city, not just the 2 hardcoded ones.

Sketch the safe-access version yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a real API call working end to end for the 2 cities the demo hardcodes, and are honest about not yet handling failures. Advanced names the 2 places where "works for the demo" and "would survive a real project" diverge — a response shape that isn't guaranteed to stay put, and a lookup table that

silently limits what the tool's own description promises — and shows what closing each gap actually looks like in code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a tool called get_weather that takes a city name

inside it:
    build the URL/params for the weather API
    call request_with_retry with that URL
    pull out the temperature from the response
    return a short text summary

register the tool on a model

ask "what's the weather in Lahore?"

print the tool call, then actually run it and print the real result
```

Here is almost the whole thing, missing the actual API call:

```
# real_api_tool_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from http_client import request_with_retry # your Doc02 client

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    # TODO: build the params, call request_with_retry, pull out the temperature
    pass

model_with_tools = ChatOpenAI().bind_tools([get_weather])
```

Fill in the `TODO` , then run it and check against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

One piece to get you unstuck — the shape of the params dict and the return value:

```
# real_api_tool_practice.py
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

def get_weather(city: str) -> str:
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return f"No coordinates on file for {city}."
    params = {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }
    # TODO: call request_with_retry(WEATHER_URL, params=params), pull out
    # data["current_weather"]["temperature"], and return a text summary.
```

Finish the `TODO` yourself, wire it up to a model with `@tool` and `bind_tools`, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the missing safe-access piece yourself — this version defends against a response that doesn't have the key you expected:

```
# real_api_tool_practice.py
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

def get_weather(city: str) -> str:
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return f"No coordinates on file for {city}."

    params = {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }
    data = request_with_retry(WEATHER_URL, params=params)

    # your turn: instead of data["current_weather"]["temperature"] (which raises
    # KeyError if the shape isn't what you expect), use .get(...) chains and
    # return a clear "malformed response" string if the temperature isn't there
    ...
```

For the geocoding half, sketch the second `request_with_retry()` call yourself — same client, different URL, turning a city name string into a `{"latitude": ..., "longitude": ...}` dict instead of looking it up in a fixed dictionary. Then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying tool (build params, call the API, return a summary) at 3 completeness levels — Basic and Intermediate trust the response shape completely and only support 2 hardcoded cities, while Advanced adds the defensive access pattern that turns a shape-mismatch crash into a clear returned message, and replaces the fixed lookup table with a real second API call so the tool can actually do what its own description promises for any city, not just the 2 it happens to know about.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)